

Data Processing Pipeline

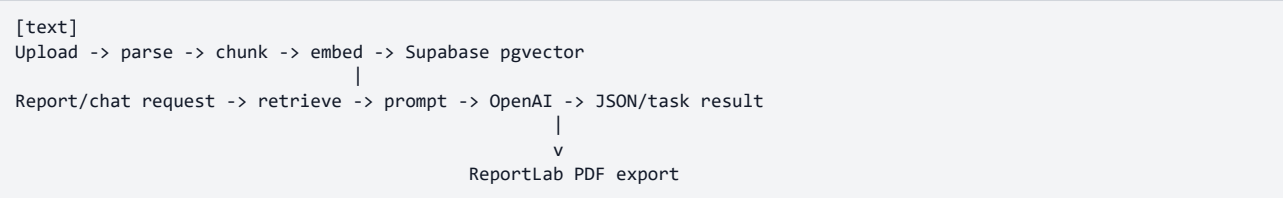
Source: docs/pipeline.md

Data Processing Pipeline

Status: internal technical documentation Last updated: 2026-05-23 Extended reference: ARCHITECTURE_DEEP_DIVE.md, SYSTEM_OVERVIEW.md

This document describes the current path from user upload to ESG report PDF.

Overview



Upload and Parsing

User document upload is handled by POST /user/documents/upload. The frontend component MultiFileUpload.jsx sends one file per request with an optional ESG tag and then polls /status/{task_id}.

The Celery task parses files through ParserDispatcher:

Format	Parser
PDF	backend/parsers/pdf_parser.py
DOCX	backend/parsers/docx_parser.py
XLSX/CSV	backend/parsers/tabular_parser.py

DOCX paragraph text becomes parse_result.text. Tables are stored separately in parse_result.tables; the current RAG path relies mainly on paragraph text, so important test data should also appear in paragraphs.

Chunking and Embeddings

Text is split by backend/ingestion/chunker.py with sentence/paragraph-aware logic and token estimates. User chunks are stored in user_document_chunks. Knowledge-base chunks are stored in knowledge_chunks.

The embedding service uses OpenAI text-embedding-3-small.

Knowledge Base

Administrators upload knowledge documents through /knowledge/upload or /knowledge/parse-and-store. Metadata lives in knowledge_documents; vector chunks live in knowledge_chunks.

The knowledge base is shared context for regulations and standards. Report generation must not use numeric indicators from legal documents as company KPIs.

Retrieval

backend/RAG/rag_retriever.py calls Supabase RPC match_chunks2 with:

- query embedding
- match threshold/count
- optional filter_tag
- query_user_id

Returned rows are sorted by similarity and formatted as:

```
[text]
--- DOKUMENT: <source> ---
<chunk_text>
```

Partial reports use tag aliases:

- Environmental, environmental, E, e
- Social, social, S, s
- Governance, governance, G, g

The full ESG report runs without a tag filter.

Report Generation

POST /report/generate queues backend.generate_report.

The Celery task:

- Retrieves RAG context.
- Splits chunks into company data and legal/knowledge-base context.
- Builds a strict JSON prompt using the selected reporting standard checklist

(GRI, SASB or TCFD; defaults to GRI for legacy requests).

- Calls OpenAI gpt-4o-mini.
- Parses the response as JSON.
- Saves report history when possible.
- Returns the JSON and used_chunks as the Celery task result.

The structured payload includes legacy fields plus optional richer fields:

- streszczenie_wykonawcze
- standard_raportowania
- zakres_i_metodyka
- szczegolowa_analiza
- luki_w_danych
- rekomendacje
- zgodnosc_ze_standardami

If retrieval finds no chunks for a selected partial scope, the task returns `partial_success` with `data: null`. The frontend still allows PDF export, and the PDF renderer produces an empty-state report.

PDF Export

GET /report/download/{task_id} reads the cached Celery task result and renders PDF through `backend/utils/pdf_generator.py`. It does not call OpenAI again.

The PDF includes:

- cover page
- running header on pages after the cover
- footer and page numbers
- executive summary, methodology and detailed analysis
- numeric KPI table
- actions, risks, data gaps, recommendations and standards alignment
- legal/compliance summary
- RAG citations from used_chunks

- PDF outline/bookmarks

The cover can include a logo through ESG_PDF_LOGO_PATH. TTF fonts are preferred for Polish glyph support.

Chat

POST /chat/ask stores the user message, queues a Celery RAG task and returns a task_id plus session_id. The frontend or API client polls /status/{task_id} for the answer task result. Session history is available through /chat/sessions and /chat/sessions/{session_id}/history.